

Prático para Desenvolvedor Front-End Pleno

Objetivo:

Avaliar sua capacidade de construir interfaces funcionais e bem organizadas com Angular e PrimeNG. O teste foca em:

- Layout responsivo e uso de componentes do PrimeNG
- Formulários reativos com validações e máscaras
- Controle de estado e reatividade com observables
- Organização do código com componentes e serviços reutilizáveis
- Aplicação de boas práticas de UX e testes unitários

A proposta simula um cenário real de desenvolvimento e busca medir tanto habilidades técnicas quanto atenção aos detalhes.

1. Reatividade, Validações e Máscaras (Reactive Forms + PrimeNG)

Teste sugerido:

Criar um formulário de cadastro de cliente com um layout responsivo (mobile-first) usando componentes do PrimeNG com os seguintes campos:

- Nome (obrigatório, mínimo 3 caracteres)
- Email (obrigatório, formato válido)
- CPF (obrigatório, máscara 999.999.999-99)
- Data de nascimento (obrigatório, não pode ser futura)
- Contato (máscara dinâmica para fixo ou celular) com opções de mais de um tipo de telefone (Residencial, Fixo, Whatsapp)
- País e Estado (ao selecionar país, os estados devem ser atualizados dinamicamente)

Requisitos técnicos a serem avaliados:

- Uso de ReactiveFormsModule (FormBuilder, FormGroup, Validators)
- Reações a valueChanges para alterar validações dinamicamente (ex: se país for "Brasil", CPF é obrigatório)
- Máscaras com p-inputMask ou bibliotecas como ngx-mask
- Uso de componentes do PrimeNG como p-calendar, p-inputText, p-dropdown
- Exibição amigável de mensagens de erro abaixo dos campos

Obs : Implemente pelo menos uma validação com uso de diretivas.

2. Listagem e Controle de Estado

- **Teste:** Criar uma página com lista paginada de clientes e filtros (Filtro por nome e cidade) que preservam estado ao navegar entre páginas. Implemente botões de "editar", "remover" e "novo cliente".
- **Objetivo:** Avaliar uso de serviços para armazenar estado, uso correto de BehaviorSubject, ou uso de NgRx.
- **Bônus:** Armazenar estado no localStorage ou URL (queryParams).

3. Componentização e Reuso

- **Teste:** Criar um componente reutilizável de modal de confirmação que aceite título, mensagem e callbacks via @Input() e @Output().
- **Objetivo:** Avaliar encapsulamento e reuso.

4. Integração com API (HTTP + interceptors)

- **Teste:** Consumir uma API REST para listar, criar, editar e deletar itens com feedback ao usuário (mensagens de sucesso/erro via MessageService) (pode ser mockada via JSON-server ou interceptor com of()).

- **Objetivo:** Avaliar HttpClient, HttpInterceptor, e tratamento de erros.

5. Observabilidade e Feedbacks

Implemente uma estratégia simples de observabilidade no contexto do cadastro de cliente. Ao realizar ações como salvar, editar ou excluir, registre eventos no console ou em um serviço de log. Além disso, apresente feedback visual ao usuário com mensagens de sucesso ou erro.

Requisitos técnicos:

- Criar um serviço de logging centralizado (ex: LogService) para registrar ações importantes
- Utilizar o MessageService do PrimeNG (p-toast) para exibir notificações ao usuário
- Registrar erros de API ou falhas de validação com contexto adequado

6. Testes Automatizados

- **Teste:** Criar testes unitários do componente de formulário e lógica condicional. (Jasmine + Karma)
- **Extra:** Teste e2e básico com Cypress (opcional).

Observações Gerais

O candidato deverá disponibilizar o código-fonte em um **repositório público** utilizando uma plataforma de versionamento como GitHub, GitLab ou Bitbucket. Além disso, deverá gravar um **vídeo** auto explicativo, apresentando o funcionamento da aplicação e justificando as principais decisões técnicas adotadas no desenvolvimento.